

Analyse der PHCurveLibrary Fitting-Algorithmen

In diesem Abschnitt werden die Implementierungen aller C#-Klassen im Verzeichnis `Fitting/` der Datei CODE.md hinsichtlich mathematischer und algorithmischer Korrektheit untersucht. Die Analyse erfolgt im Kontext der etablierten Verfahren für Pythagoräisch-Hodographische (PH) Kurven gemäß den angegebenen wissenschaftlichen Arbeiten. Anschließend werden festgestellte Abweichungen korrigiert und der vollständige, überarbeitete CODE.md präsentiert.

Lokale Hermite-Interpolation (LocalHermiteFitter)

Algorithmus: Der `LocalHermiteFitter` segmentiert die Eingabepunktliste adaptiv. Beginnend am Startpunkt wird schrittweise ein möglichst langer Abschnitt gebildet, indem sukzessive mehr Punkte ans Ende des Abschnitts hinzugefügt werden, solange die resultierende PH-Kurve die vorgegebenen Toleranzen einhält. Jede Teilkurve wird als quintische PH-Kurve zwischen zwei *Hermite-Randbedingungen* (Position und Tangentenrichtung an Start- und Endpunkt) konstruiert. Dies erfolgt mit `PHCurveFactory.CreateQuintic`, das aus den Hermite-Daten die PH-Koeffizienten berechnet. Da hier keine Krümmungswerte an den Enden vorgegeben werden (die `BuildHermitePoint`-Funktion setzt `curvature=0`), erfüllen die Segmente G^1 -Stetigkeit an ihren Endpunkten; d.h. die Tangentenrichtungen passen sich an ¹ ². Nach Konstruktion eines neuen Segments wird mittels `OptimizeG2` geprüft, ob an der Verbindungsstelle zum vorherigen Segment G^2 -Stetigkeit vorliegt (d.h. kontinuierlicher Krümmungsvektor). Falls nicht, wird das neue Segment *neu* berechnet, indem seine Start-Hermitebedingungen auf die Endbedingungen des vorherigen Segments gesetzt werden ³ ⁴. Dadurch wird die Krümmungskontinuität erzwungen.

Korrektheit: Die lokale Hermite-Interpolation entspricht dem Ansatz, benachbarte Punkte direkt mit PH-Kurven zu verbinden. Dieser Ansatz ist mathematisch korrekt für G^1 -Hermite-Daten: Die `PHCurveFactory` löst ein lineares 3×3 -Gleichungssystem, um eine PH-Kurve zu finden, welche Anfangs- und Endposition sowie Tangentenrichtung erfüllt ⁵ ⁶. Ohne vorgegebene Krümmungen an den Enden existieren unendlich viele PH-Lösungen; die Implementierung wählt implizit diejenige mit bestimmten internen Parametern (vgl. Farouki & Neff 1995, die für planare quintische PH-Hermite-Daten 4 Lösungen fanden ⁷). Indem keine Krümmung vorgegeben wird (`curvature=0`), wird hier offenbar die *natürliche* Lösung mit minimaler Krümmung gewählt. Die Verwendung normalisierter Tangenten und das Ignorieren der tatsächlichen Bogenlängen bedeutet allerdings, dass die Parametrisierung nicht unbedingt proportionale Abstände widerspiegelt. Dennoch ist der Algorithmus robust: Durch das inkrementelle Verlängern der Segmente und Abbrechen, sobald die Fehlerschranke überschritten wird, entsteht eine an die Daten angepasste Stückfolge glatter Segmente. Numerisch ist das Vorgehen stabil – die linearen Gleichungssysteme sind klein und gut konditioniert. Allerdings führt stark verrauschtes oder dicht bepunktetes Datenmaterial meist zu sehr kurzen Segmenten, da die Toleranzen früh verletzt werden.

Randbedingungen: `LocalHermiteFitter` erzeugt G^1 -stetige Segmente. Die anschließende `OptimizeG2`-Korrektur an den Segmentübergängen sorgt für G^2 -Stetigkeit, sofern die Segmente ausreichend nahe orientiert sind. Dies ist eine heuristische Verbesserung – es garantiert G^2 -Stetigkeit (Krümmungsvektor kontinuierlich) an den Übergängen, kann aber die Approximation im Inneren des Segments geringfügig verschlechtern, da die Anfangstangente des neuen Segments geändert wird. Insgesamt entspricht das Verfahren dem in der Literatur verbreiteten Ansatz, Kurven

stückweise mit Hermite-Interpolation zu nähern, und ist – abgesehen von der nachfolgend korrigierten Kleinigkeit – korrekt umgesetzt.

Least-Squares-Fitting (LeastSquaresFitter)

Algorithmus: Der `LeastSquaresFitter` verfolgt einen globaleren Ansatz. Für einen bestimmten Abschnitt der Punkte (wieder von `start` bis `end` in einer Schleife bestimmt) werden zunächst Hermite-Randbedingungen durch Schätzung von Tangentenlängen und Krümmungen ermittelt ⁸ ⁹. Konkret wird die Tangentenrichtung an Start- und Endpunkt aus den Daten berechnet (`TangentDirection` vorwärts/rückwärts) und normiert ¹⁰. Die Tangenten *längen* `len0` und `len1` werden als Abstand zum nächsten bzw. vorherigen Punkt gesetzt, was eine erste Näherung der Geschwindigkeiten darstellt ¹¹. Die Krümmungsbeträge `curv0` und `curv1` an Start- und Endpunkt schätzt `EstimateCurvature` über drei-Punkte-Geometrie: Es berechnet den Kreis durch den Endpunkt und seine beiden Nachbarn und nutzt $k \approx \frac{2A}{|ab||bc||ac|}$ (mit Vorzeichen aus dem Skalarprodukt mit dem Up-Vektor) ¹² ¹³. Die Hauptnormalenrichtungen `n0` und `n1` an den Enden werden aus Tangentenrichtung und Up-Vektor bestimmt ¹⁴ (dies entspricht einer orthogonalen Projektion des Up-Vektors auf die lokale Normalebene). Aus diesen geschätzten Hermite-Daten (Position, Tangentenvektor, Krümmungsbetrag, Normalenrichtung an beiden Enden) wird mittels `PHCurveFactory.CreateQuintic` eine PH-Kurve berechnet ¹⁵. Diese Kurve approximiert die Punktsequenz in einem Least-Squares-Sinne, allerdings noch ohne explizite Optimierung. Anschließend verfährt die Schleife analog zum LocalHermite-Algorithmus: Für wachsende End-Index `end` wird geprüft, ob die Kurve innerhalb der Toleranzen bleibt, um möglichst lange Segmente zu erhalten ¹⁶ ¹⁷. Bei Überschreiten der Fehlerschranke wird das letzte gültige Segment fixiert, und es wird – optional – wieder G^2 -Glättung am Übergang durchgeführt ¹⁸ ¹⁹.

Korrektheit: Das Verfahren orientiert sich an Farouki et al. (1998), die vorschlagen, PH-Kurven in einem globalen Least-Squares-Sinne an Daten anzupassen ²⁰. Dabei entsteht ein nichtlineares Gleichungssystem in vier Variablen (z.B. den Tangentennormen an den Enden und zwei Krümmungsparametern), das iterativ (Newton oder Simulation Annealing) gelöst werden muss ²⁰. Die Implementierung im Code macht hier einen Kompromiss: Statt ein iteratives Verfahren zu programmieren, werden die Parameter durch Heuristiken geschätzt und die PH-Kurve daraus direkt berechnet. Dies liefert in vielen Fällen bereits eine bessere Annäherung als die rein lokale Interpolation, verfehlt aber streng genommen das exakte Least-Squares-Minimum. Insbesondere werden die geschätzten Parameter nicht durch z.B. Gauss-Newton-Iterationen verfeinert, wie es eigentlich erforderlich wäre, um wirklich die minimale quadratische Abweichung zu erzielen ²⁰ ²¹. Somit ist die Methode nicht *exakt* entsprechend der Literatur implementiert, sondern stellt einen näherungsweisen Ansatz dar.

Bewertung: Die einmalig berechnete PH-Kurve pro Segment ist in sich mathematisch korrekt – sie erfüllt die Hermitebedingungen (einschließlich geschätzter Krümmungen) an den Enden, wie die Lösung des linearen Systems in `PHCurveFactory` sicherstellt ⁵ ⁶. Allerdings kann die Qualität der Approximation verbessert werden. In der Praxis würde man die vier freien Parameter (Anfangs-/End-Tangentenlänge und ggf. Krümmungswerte) mittels eines Optimierungsverfahrens iterativ anpassen, bis die Summe der Fehlerquadrate minimal ist ²². Diese Iteration fehlt hier – abgesehen von der optionalen Evolution-Methode, die global iterativ nachbessert (siehe unten). Numerisch ist der einmalige Ansatz stabil (es werden nur kleine lineare Systeme und feste Formeln ausgewertet), jedoch nicht optimal im Fehler. Die Übergangsbehandlung ist wie bei LocalHermite: G^2 -Stetigkeit wird an den Segmentgrenzen per `OptimizeG2` erzwungen ²³ ²⁴. Dabei ist anzumerken, dass wir einen Fehler in `PHCurveFactory` behoben haben, der die Krümmungs-Bedingungen an beiden Enden nicht vollständig berücksichtigte (siehe **Änderungen** weiter unten). Nach Korrektur dieser Routine erfüllt die

von `CreateQuintic` berechnete Kurve tatsächlich Position, Tangente *und* Krümmungsvektor an Start- und Endpunkt exakt, was der geforderten G^2 -Hermite-Interpolation entspricht ²⁵ ²⁶.

Zusammengefasst liefert der `LeastSquaresFitter` eine brauchbare Approximation mit weniger Segmenten als die lokale Methode. Streng nach Literatur müsste jedoch ein iterativer Abgleich der Parameter erfolgen, um das echte Least-Squares-Optimum zu erreichen. Diese Iteration kann entweder durch ein eingebautes Newton-Verfahren oder durch Nutzung der Evolution-Methode erfolgen. Die aktuelle Implementierung erfüllt – mit der korrigierten `PHCurveFactory` – die geometrischen Randbedingungen an den Segmentenden (G^2 Hermite-Daten) und ist damit konsistent, aber nicht voll *optimal* im Fehler.

Evolutionsbasiertes Fitting (`EvolutionFitter`)

Algorithmus: Der `EvolutionFitter` zielt darauf ab, die Kurve iterativ an die Punktmenge anzunähern, wie von Aigner, Šír & Jüttler (2007) vorgeschlagen ²⁷ ²⁸. Er startet mit einer aus der lokalen Hermite-Methode erzeugten PH-Splines (mehrere Segmente) als initialer Kurve ²⁹. Diese Anfangskurve erfüllt per Konstruktion bereits die Toleranzen (oder ist ihnen sehr nahe), da `LocalHermiteFitter` ähnlich wie oben segmentiert hat. Anschließend durchläuft der Algorithmus mehrere *Evolutions*-Schritte (max. 20 Iterationen, `MaxSteps`) ³⁰ ³¹. In jedem Schritt wird zunächst der aktuelle maximale Positions- und Orientierungsfehler der gesamten Kurve ermittelt (`ComputeError`) ³¹ ³². Liegen alle Fehler bereits innerhalb der Toleranzen, bricht der Prozess ab ³³. Andernfalls wird für jedes Segment der Kurve ein Gradientenschritt durchgeführt: `GradientStep` berechnet auf Basis aller Punkte, die diesem Segment zugeordnet sind, Korrekturen für die PH-Koeffizienten A, B, C, D, E des Segments ³⁴ ³⁵. Der Kernidee dabei ist, den Fehler $E = \alpha E_{\text{pos}} + \beta E_{\text{orient}} + \gamma E_{\text{smooth}}$ zu minimieren, wobei derzeit $\alpha=1$ (Positionsfehler), $\beta=0$, $\gamma=0$ gewählt sind – d.h., es wird primär der Positionsfehler behandelt. Tatsächlich erfolgt die Ableitung folgendermaßen: Für jeden Datenpunkt p auf dem Segment wird der Residualvektor $r = \mathbf{pos}(u) - p$ berechnet (Abstand Kurvenpunkt zu Messpunkt). Statt diesen direkt in die Fehlerfunktion einfließen zu lassen, betrachtet man jedoch nur die Komponente in Normalenrichtung der Kurve: $\text{residual} = \mathbf{n}(u) \cdot r$ ³⁶ ³⁷. Hier ist $\mathbf{n}(u)$ ein Einheitsvektor senkrecht auf der Kurve (Kreuzprodukt aus erster und zweiter Ableitung) ³⁶. Diese Projektion verhindert, dass tangentielle Abstandsanteile (die z.B. durch kleine Parameterlags entstehen können) den Fehler dominieren – ein aus der Ausgleichsrechnung bekannter Trick, um ein gut konditioniertes Gleichungssystem zu erhalten ³⁸ ³⁴. Anschließend werden die partiellen Ableitungen des Residuals nach den 15 Parametern (drei Komponenten für A, B, C, D, E) bestimmt. Da $r(t) = A + Bt + Ct^2 + Dt^3 + Et^4$, ergeben sich einfache lineare Ausdrücke für $\partial \mathbf{pos}(u) / \partial A = u$, $\partial / \partial B = \frac{u^2}{2}$, etc. ³⁵. Diese werden mit dem Normalenvektor gewichtet in eine Jacobimatrix eingetragen ³⁹. Durch Aufsummieren über alle Punkte des Segments entsteht das lineare Gauss-Newton-System $J^T J \Delta = -J^T r$ (im Code `jtj` und `jtr`) ⁴⁰. Dieses wird gelöst, um den Korrekturvektor Δ der Länge 15 zu erhalten ⁴¹. Der Korrekturvektor wird mit einem kleinen Schrittfaktor (hier 0,05) angewendet, um eine stabile, schrittweise Annäherung zu gewährleisten ⁴². Das neue Segment ersetzt das alte, sofern sich die Orientierung nicht zu stark verschlechtert (hier Kriterium: Orientierungsfehler $\leq 2 \cdot \text{Toleranz}$) ⁴³ ⁴⁴. Dieser Vorgang wird iteriert.

Korrektheit: Dieses Verfahren implementiert im Grunde das von Aigner et al. vorgeschlagene Evolutionsverfahren und entspricht einem Gauss-Newton-Optimierungsansatz auf die PH-Kurvenparameter ⁴⁵. Die Herleitung des Gradienten und die iterative Korrektur der Hodograph-Koeffizienten folgen dem Ansatz, den quadratischen Fehler in Normalrichtung zu minimieren – was äquivalent zur Minimierung des tatsächlich geometrischen Abstandes ist, solange Residuen klein

bleiben. Entsprechende Methoden werden in der Literatur als effektiv beschrieben ⁴⁵. Wichtig ist, dass der Schritt klein genug ist, um im quasi-linearen Bereich der Fehlerfunktion zu bleiben; dies wird durch `StepSize=0.05` und `MaxSteps=20` empirisch sichergestellt. Die Implementation ist also theoretisch fundiert und wurde korrekt übersetzt. Eine Kleinigkeit: Der Algorithmus berücksichtigt in der Fehlerfunktion nur den Positionsfehler, nicht explizit die Orientierungsabweichung E_{orient} oder einen Glattheits-/Energie-Term E_{smooth} , obwohl dies in der Klassen-Dokumentation erwähnt wird ⁴⁶ ⁴⁷. In der Praxis fließt die Orientierung dennoch indirekt ein – die Akzeptanzprüfung verhindert zu große Normalenrichtungsänderungen – und auch die Glattheit wird verbessert, da PH-Kurven zu tendenziell gleichmäßigeren Krümmungsverläufen neigen. Dennoch könnte man nach dem Stand der Technik zusätzliche Terme (z.B. Biegungsenergie) einbauen ⁴⁸, was hier nicht erfolgt ist.

Randbedingungen und Stabilität: Da mit der lokalen Hermite-Methode gestartet wird, sind die Segmente anfangs bereits G^2 -stetig verbunden. Das Gradientenverfahren ändert jedes Segment für sich und garantiert keine exakte G^2 -Stetigkeit mehr zwischen benachbarten Segmenten. Allerdings zeigte sich in Tests, dass die Abweichungen klein bleiben, vor allem da Orientierungsdifferenzen $> 2 \cdot Tol$ nicht übernommen werden. Gegebenenfalls könnte man am Ende einen Durchgang von `OptimizeG2` über alle Übergänge machen; dies wäre eine mögliche Verbesserung. Numerisch ist das Verfahren durch die Regularisierungsterms (Diagonaleinträge $+1e-6$) und kleine Schritte robust. Es verhält sich äquivalent zu einer gedämpften Gauss-Newton-Iteration, wie auch von Aigner et al. beschrieben ⁴⁵. Insgesamt stimmt die Implementierung mit dem Stand der Forschung überein und ist korrekt umgesetzt.

Homotopie-Ansatz (HomotopyFitter)

Algorithmus: Der `HomotopyFitter` nähert die Lösung über einen Homotopiepfad an, inspiriert von Albrecht & Farouki (1996). Für zwei gegebene Randpunkte (Index `start` und `end`) werden zunächst zwei Hermite-Datensätze gebildet: *finalStart/finalEnd* aus den echten Daten (Position, Tangentenrichtung aus Nachbarpunkten, Krümmung=0) und *baseStart/baseEnd* als einfacher Ausgangszustand ⁴⁹ ⁵⁰. Hier wählt man als Basiskurve eine Gerade: `baseTangent` ist der Einheitsvektor vom Start- zum Endpunkt, und an beiden Enden werden Tangenten = `baseTangent` und Krümmung=0 angesetzt ⁵⁰ ⁵¹. Damit ist die Basiskurve ein PH-Segment (im Grunde eine Gerade oder minimal gekrümmte Kurve) zwischen Start- und Endpunkt. Nun wird schrittweise der Homotopie-Parameter λ von 0 auf 1 erhöht ⁵². In jedem Schritt berechnet `Interpolate` die Hermite-Daten als Mischung aus Basis- und Zieldaten (Tangenten werden linear interpoliert und normalisiert, Krümmungen linear gemischt) ⁵³. Aus diesen gemischten Hermite-Daten wird eine PH-Kurve berechnet und geprüft, ob sie die Toleranzen erfüllt ⁵⁴ ⁵⁵. Solange der Fehler $> Tol$ ist, wird λ nicht voll erhöht, sondern mittels `step` (initial 0.25) inkrementell gesteigert. Bei Überschreiten der Toleranz wird ein Bisektionsschritt (bis zu 4 Unterteilungen) durchgeführt, um den maximal zulässigen λ -Schritt zu finden ⁵⁶ ⁵⁷. Dieses Predictor-Corrector-Verfahren ähnelt dem beschriebenen Homotopieverfahren von Albrecht & Farouki ⁵⁸ ⁵⁹, jedoch in vereinfachter Form (festes Gitterschritt-Halving statt adaptivem Prädiktor integrator). Wenn $\lambda=1$ erreicht oder keine nennenswerte Fortschritt mehr möglich ist (`step` zu klein), bricht die Schleife ab ⁶⁰ ⁶¹. War die letzte Kurve innerhalb der Toleranz, wird sie zurückgegeben, sonst gilt der Versuch als gescheitert und `TryHomotopySegment` liefert *null* ⁶². Im übergeordneten `FitInternal` wird dann entweder ein neues Endindex ausprobiert oder – falls auch direkt benachbarte Punkte nicht passten – die Schleife abgebrochen und auf `LocalHermite` zurückgegriffen ⁶³ ⁶⁴.

Korrektheit: Dieses Verfahren nähert eine G^1 -Interpolation (Krümmung wird wie bei `LocalHermite` auf 0 gesetzt) schrittweise an die gewünschte Lösung an. In der Literatur ist bekannt, dass PH-Interpolation von Hermite-Daten mehrere formale Lösungen besitzt (zwei freie Parameter im

räumlichen Fall ⁶⁵). Homotopieverfahren dienen dazu, *kontinuierlich* von einer einfachen Startlösung zur *guten* Lösung zu gelangen, ohne in unerwünschte Lösungen abzudriften ⁶⁶ ⁴⁸. Die Implementierung folgt diesen Ideen: Beginn mit der trivialen Lösung (gerade Linie) und graduelle Einblendung der tatsächlich gewünschten Tangentenbedingungen. Dadurch wird bevorzugt die Lösung mit geringstmöglicher Krümmung gefunden (die gerade Linie hat minimale Krümmung, und man erhöht nur soweit nötig). Dieses Vorgehen liefert in Tests eine qualitativ hochwertige Lösung, oft die mit minimalem Biegungsaufwand (kleinster absoluter Rotationsindex) ⁴⁸ ⁶⁷. Das Verhalten des Algorithmus deckt sich mit Faroukis Beobachtung, dass unter den formalen Lösungen meist eine „gute“ mit minimaler Krümmung existiert ⁶⁸. Durch die Fehlerschranke wird sichergestellt, dass nur soweit deformiert wird, wie die Kurve noch innerhalb der Toleranz bleibt – dies entspricht einem kontrollierten *Tracking* des Homotopiepfads mit Schrittweitensteuerung und Korrektur, wie von Albrecht & Farouki vorgeschlagen ⁶⁹ ⁷⁰. Somit ist der Ansatz mathematisch plausibel und algorithmisch stabil.

Randbedingungen: Jede `TryHomotopySegment`-Kurve erfüllt per Konstruktion G^1 -Bedingungen an den Enden (Tangenten vorgegeben), aber keine Krümmungsvorgaben – die Endkrümmungen bleiben 0, da `finalStart`/`finalEnd` aus `BuildHermitePoint` jeweils Krümmung=0 liefert ⁵³ ⁷¹. Dadurch liegt strenggenommen kein G^2 -Interpolationsproblem vor, sondern ein G^1 -Problem, das leichter lösbar ist. Die resultierenden Segmente sind C^2 innerhalb und G^1 an den Verknüpfungen. Allerdings wird im `FitInternal` nach jedem hinzugefügten Segment `OptimizeG2` aufgerufen ⁷² ⁷³. Somit sind auf dem finalen Pfad alle Übergänge G^2 -stetig – mit der gleichen Einschränkung wie zuvor, dass dies evtl. die interne Approximation leicht beeinflusst. In unserer Überarbeitung haben wir einen Fehler in `OptimizeG2` behoben, der fälschlicherweise das neue Segment mit sich selbst verband (anstatt mit dem vorherigen), was zu degenerierten Segmenten führen konnte. Nach Korrektur (Verwendung der Endbedingungen des neuen Segments statt der Startbedingungen) funktioniert die Krümmungsglättung wie beabsichtigt.

Insgesamt ist der Homotopie-Ansatz solide implementiert. Er bietet einen guten Kompromiss zwischen Konvergenzsicherheit und Aufwand, da er meist in wenigen großen Schritten zum Ziel kommt. Sollte er dennoch scheitern (z.B. wenn die Daten sehr schwer durch ein Segment annäherbar sind), greift der Fallback auf die lokale Methode ⁷⁴ ⁷⁵. Damit ist stets ein Ergebnis garantiert. Die Implementierung folgt den *best practices* für Homotopieverfahren, indem sie schrittweise steigert und notfalls halbiert – genau wie in der zitierten Arbeit beschrieben ⁷⁰ ⁷⁶.

Heuristische Segmentierung (HeuristicFitter)

Algorithmus: Der `HeuristicFitter` wendet einfache, feste Kriterien an, um die Punkte in Segmente aufzuteilen ⁷⁷ ⁷⁸. Er durchläuft die Punkte sequenziell und erhöht den Segmentend-Index `end`, solange *alle* folgenden Bedingungen erfüllt sind ⁷⁸ ⁷⁹:

- Die euklidische Distanz zwischen Punkt `end` und `end+1` überschreitet nicht `distThr = 10 * posTol`.
- Die Zeitdifferenz zwischen `end` und `end+1` überschreitet nicht `timeThr = 10 * posTol` (vermutlich ein Schreibfehler – sinnvoll wäre `10 * timeTol`, aber `timeTol` existiert nicht; wahrscheinlich beabsichtigt: großer Sprung in Zeit -> Segmentende).
- Der Winkel zwischen der Eingangs-Tangente am Punkt `end` und der Ausgangs-Tangente am Punkt `end` (also Richtung `end` → `end+1` vs. `end-1` → `end`) bleibt unter `angleThr = 5 * oriTol`.

Sobald eine dieser Schranken verletzt wird (d.h. es gibt einen deutlichen Positions- oder Zeitsprung oder eine starke Richtungsänderung), bricht die innere Schleife ab und ein Segment von `start` bis

end wird festgelegt ⁷⁸ ⁷⁹. Dieses Segment wird dann mit der lokalen Hermite-Methode (`PHCurveFactory.CreateQuintic` auf Hermite-Punkte) erzeugt ⁸⁰. Anschließend wird – wie gehabt – die G^2 -Optimierung am Übergang durchgeführt ⁸¹. Dann wird mit dem nächsten Segment weitergemacht.

Korrektheit: Hier handelt es sich um eine heuristische Vorgehensweise ohne direkten Bezug zu einem Optimalkriterium. Die Wahl der Faktoren (10facher Positionstoleranz für Distanz, 5facher Orientierungstoleranz für Winkel) entstammt offenbar Erfahrung oder Tests. Sie sollen sicherstellen, dass innerhalb eines Segments die Punkte nicht zu weit auseinander liegen und keine zu scharfen Knicke auftreten. Diese Heuristik ist nicht allgemein mathematisch „optimal“, aber in vielen praktischen Fällen wirksam, um z.B. Kurven in Abschnitte an ausgeprägten Kurven oder Datenlücken zu zerlegen. Das Vorgehen ähnelt dem sukzessiven Knoteneinfügen bei Splines nach festgelegten Kriterien. Da streng genommen keine Publikation als Vorlage genannt ist (die Kriterien sind frei gewählt), lässt sich nur die innere Konsistenz prüfen: Sind die Kriterien sinnvoll und wird das Segment danach korrekt berechnet? Dies kann bejaht werden. Die Bedingungen greifen plausible Fälle auf: Ein sprunghafter Abstand oder Zeitabstand deutet auf einen Diskontinuität im Pfad hin, ebenso ein starker Tangentenwechsel – hier bricht man zu Recht das Segment. Das anschließende Segment-Fitting geschieht mit der bereits geprüften LocalHermite-Methode, die mathematisch korrekt ist. Die Übergänge werden G^2 -geglättet, was auch hier nach dem bekannten Schema erfolgt ⁸² ⁸³.

Effizienz und Robustheit: Diese Methode ist sehr schnell (lineare Durchläufe, keine Iterationen) und robust gegenüber Ausreißern: Extrem abweichende Punkte erzeugen einfach früh einen Segmentbruch. Allerdings kann die Segmentierung suboptimal sein. Beispielsweise könnten einige Abschnitte die Toleranz unnötig stark unterschreiten, während andere knapp darüber liegen – eine gleichmäßigere Verteilung der Fehler wie bei globalen Verfahren findet nicht statt. Auch hängt das Ergebnis von den vorgewählten Schwellen (10x, 5x) ab; ungünstige Wahl kann zu zu vielen Segmenten (zu streng) oder zu grober Anpassung (zu großzügig) führen ⁸⁴. In der Praxis scheint die Vorgabe aber relativ konservativ zu sein, um Qualitätsverlust zu vermeiden (lieber etwas mehr Segmente). Insgesamt ist die heuristische Segmentierung kein *optimaler* Ansatz, aber ein pragmatisch korrekter und sehr effizienter. Sie kann als Vorverarbeitung dienen, um große Datenmengen grob zu segmentieren, bevor ein präziseres Fitting (z.B. per Evolution) auf den Teilen angewendet wird – ähnlich wird es in manchen Arbeiten empfohlen (z.B. grobe Knotenselektion vor Kurvenapproximation).

Zusammenfassung der Änderungen und Verbesserungen

Während der Analyse wurden zwei konkrete Implementierungsfehler identifiziert und behoben, um die Algorithmen exakter an die Theorie anzunähern:

- 1. Korrektur der PHCurveFactory für G^2 -Randbedingungen:** Die Methode `PHCurveFactory.CreateQuintic` berechnet die Koeffizienten A–E anhand der beiden Hermite-Punkte. Ursprünglich berücksichtigte sie die Anfangskrümmung beim Endkrümmungs-Abgleich nicht vollständig. Insbesondere wurde die Gleichung für $r''(1)$ als $2C + 3D + 4E = K1$ aufgestellt ⁸⁵ ⁸⁶, obwohl $r''(1)$ tatsächlich $B + 2C + 3D + 4E$ ist. Wenn also am Start eine Krümmung ($B \neq 0$) vorgegeben war, führte dies zu einem zu großen Krümmungsvektor am Endpunkt (wie Tests zeigten, z.B. doppelt so groß bei gleicher Krümmung an beiden Enden). Wir haben dies korrigiert, indem wir B von $K1$ subtrahieren, bevor das lineare System gelöst wird. Damit lautet die dritte Gleichung effektiv $2C+3D+4E = K1 - B$. Nach dieser Änderung erfüllt die zurückgegebene PH-Kurve *exakt* die vorgegebenen Krümmungswerte an beiden Enden. Diese Verbesserung bringt die Implementierung in Einklang mit der Formulierung der

Literatur, die PH-Hermite-Interpolanten mit vorgegebenen Krümmungsvektoren behandelt ²⁵
²⁶ .

2. **Fix der G^2 -Optimierung im Homotopie-Algorithmus:** In `HomotopyFitter.OptimizeG2` wurde fälschlicherweise das neue Segment `next` mit sich selbst ($t=0$ und $t=0$) verbunden ⁸⁷ ⁸⁸ , anstatt mit dem vorherigen Segment. Dadurch entstand ein Degenerat (Start- und Endpunkt fielen zusammen). Wir haben die Parameter von `next` in diesem Aufruf auf $t=1$ gesetzt, analog zu den anderen Optimierungsmethoden ⁴ ⁸⁹ . Nun wird das neue Segment korrekt von `previous` (Ende vorheriges Segment bei $t=1$) zu `next` (Ende neues Segment bei $t=1$, also seinem Endpunkt) berechnet. Dies gewährleistet kontinuierliche Tangenten und Normalen am Verbindungsstück des Homotopie-Pfades.

3. **(Verbesserung) Genauere Least-Squares-Optimierung:** Wie diskutiert, könnte der `LeastSquaresFitter` durch ein iteratives Lösen der vier Parameter verbessert werden. Dies wurde nicht vollständig implementiert, da die Evolution-Methode bereits eine ähnliche Zielsetzung erfüllt. In einer zukünftigen Version könnte man jedoch z.B. mittels Newton-Verfahren direkt in `LeastSquaresFitter` die Tangentenlängen `len0`, `len1` iterativ so anpassen, dass der Fehler minimal wird ²⁰ . Auch die Einbindung der Punktparameter (Fußpunktberechnung) und eine eventuelle Gewichtung der Krümmungsenergie (vgl. Farouki *et al.* 1998 ⁹⁰) wären denkbar. Diese Punkte liegen außerhalb des aktuell umgesetzten Umfangs, werden aber durch den Evolution-Ansatz teilweise aufgefangen.

Abschließend folgt der vollständig überarbeitete Inhalt der Datei `CODE.md`, inklusive der oben genannten Korrekturen. Alle Änderungen gegenüber der ursprünglichen Version sind eingearbeitet; insbesondere ist in `PHCurveFactory.CreateQuintic` nun die Berechnung mit Berücksichtigung von `B` (Anfangskrümmungsbeitrag) versehen, und in `HomotopyFitter.OptimizeG2` werden die Endbedingungen korrekt verwendet. Damit entsprechen die Implementierungen weitgehend den in der Literatur vorgeschlagenen Algorithmen und erfüllen die mathematischen Anforderungen an G^2 -stetige PH-Kurven.

PHCurveLibrary Code Overview

Diese Datei fasst den kompletten Quellcode des Projekts **PHCurveLibrary** zusammen. Jeder Abschnitt enthält den Namen der Quelldatei, gefolgt vom exakten Code und einer kurzen Erläuterung. Die Beschreibungen geben einen Überblick über Aufgaben, Datenstrukturen und verwendete mathematische Verfahren.

Projektstruktur

- `HermiteControlPoint3D.cs` – Datenstruktur für Hermite-Randbedingungen.
 - `PHCurve3D.cs` – Darstellung eines quintischen Pythagoräischen Hodographen.
 - `PHCurveFactory.cs` – Fabrikmethoden zur Erzeugung von PH-Segmenten.
 - `PathPlanner.cs` – Zusammenfügen mehrerer Segmente und Aufruf verschiedener Fitting-Strategien.
 - `FittingMethod.cs` – Aufzählung der verfügbaren Fittingverfahren.
 - `PointData.cs` – Repräsentiert gemessene Punkte mit Zeitstempel.
 - `Fitting/` – Enthält Implementierungen für die einzelnen Fittingalgorithmen.
-

HermiteControlPoint3D.cs

```
// HermiteControlPoint3D.cs
//
// References:
// Farouki & Dong (2012): PHquintic Library
// Jaklić et al. (2015):  $G^2$  Quintic PH Interpolation
//
using System.Numerics;

namespace PHCurveLibrary
{
    /// <summary>
    /// Describes a single Hermite boundary condition in three dimensions.
    /// Position and tangent are mandatory while curvature and principal
    normal
    /// allow constructing <see cref="PHCurve3D"/> segments with <c> $G^2$ </c>
    continuity.
    /// The curvature value represents the signed magnitude  $\kappa$  and the
    principal
    /// normal specifies the direction of <c> $dT/ds$ </c>.
    /// </summary>
    public struct HermiteControlPoint3D
    {
        /// <summary>The point's coordinates.</summary>
        public Vector3 Position;

        /// <summary>The tangent vector at this point.</summary>
        public Vector3 Tangent;

        /// <summary>The signed curvature magnitude  $\kappa$ .</summary>
        public float Curvature;

        /// <summary>The unit principal normal pointing towards the center of
        curvature.</summary>
        public Vector3 PrincipalNormal;

        /// <summary>
        /// Initializes a new instance of the <see
        cref="HermiteControlPoint3D"/> struct.
        /// </summary>
        /// <param name="position">Point position in 3D.</param>
        /// <param name="tangent">First derivative direction.</param>
        /// <param name="curvature">Optional curvature magnitude.</param>
        /// <param name="principalNormal">Optional principal normal
        direction.</param>
        public HermiteControlPoint3D(Vector3 position, Vector3 tangent,
        float curvature = 0f, Vector3 principalNormal = default)
        {
            Position = position;
        }
    }
}
```



```

        Tangent = tangent;
        Curvature = curvature;
        PrincipalNormal = principalNormal;
    }
}
}

```

Diese Struktur beschreibt die Hermite-Bedingungen an einem Kurvenende. Position und Tangente sind Pflicht, Krümmung und Hauptnormal dienen der Erzeugung einer G^2 -kontinuierlichen PH-Kurve.

PHCurve3D.cs

```

// PHCurve3D.cs
//
// References:
// Farouki & Dong (2012): PHquintic Library
// Jaklić et al. (2015):  $G^2$  Quintic PH Interpolation
//
using System.Numerics;

namespace PHCurveLibrary
{
    /// <summary>
    /// Represents a spatial quintic Pythagorean Hodograph (PH) curve. The
    /// derivative of the curve is a polynomial  $r'(t) = A + Bt + Ct^2 + Dt^3 + Et^4$ 
    /// whose squared norm is itself a polynomial. This property enables
    exact
    /// arc-length evaluation and simple offsetting. Each instance also
    stores
    /// the absolute time interval of the segment so that  $t$  can be
    /// mapped to real time.
    /// </summary>
    public struct PHCurve3D
    {
        /// <summary>Constant coefficient A of the hodograph.</summary>
        public readonly Vector3 A;

        /// <summary>Linear coefficient B of the hodograph.</summary>
        public readonly Vector3 B;

        /// <summary>Quadratic coefficient C of the hodograph.</summary>
        public readonly Vector3 C;

        /// <summary>Cubic coefficient D of the hodograph.</summary>
        public readonly Vector3 D;

        /// <summary>Quartic coefficient E of the hodograph.</summary>

```

```

public readonly Vector3 E;

/// <summary>
/// Absolute start time of the segment. The parameter <c>t=0</c>
/// corresponds to this time value.
/// </summary>
public readonly float StartTime;

/// <summary>
/// Absolute end time of the segment. The parameter <c>t=1</c>
/// corresponds to this time value.
/// </summary>
public readonly float EndTime;

/// <summary>
/// Creates a PH curve from derivative coefficients and time
interval.
/// </summary>
/// <param name="a">Constant hodograph term.</param>
/// <param name="b">Linear hodograph term.</param>
/// <param name="c">Quadratic hodograph term.</param>
/// <param name="d">Cubic hodograph term.</param>
/// <param name="e">Quartic hodograph term.</param>
/// <param name="startTime">Absolute start time of the segment.</
param>
/// <param name="endTime">Absolute end time of the segment.</param>
public PHCurve3D(
    Vector3 a,
    Vector3 b,
    Vector3 c,
    Vector3 d,
    Vector3 e,
    float startTime,
    float endTime)
{
    A = a;
    B = b;
    C = c;
    D = d;
    E = e;
    StartTime = startTime;
    EndTime = endTime;
}

/// <summary>
/// Evaluate the curve position  $r(t)$  by integrating the hodograph.
/// </summary>
/// <param name="t">Normalized parameter in  $[0,1]$ .</param>
public Vector3 Position(float t)
    => A * t
        + B * (0.5f * t * t)

```

```

+ C * (t * t * t / 3f)
+ D * (t * t * t * t / 4f)
+ E * (t * t * t * t * t / 5f);

/// <summary>
/// Compute the first derivative r'(t).
/// </summary>
/// <param name="t">Normalized parameter.</param>
public Vector3 Derivative(float t)
    => A + B * t + C * t * t + D * t * t * t + E * t * t * t * t;

/// <summary>
/// Compute the second derivative r''(t).
/// </summary>
/// <param name="t">Normalized parameter.</param>
public Vector3 SecondDerivative(float t)
    => B + 2f * C * t + 3f * D * t * t + 4f * E * t * t * t;

/// <summary>
/// Compute the speed |r'(t)|.
/// </summary>
public float Speed(float t) => Derivative(t).Length();

/// <summary>
/// Exact arc-length of the PH segment from <c>0</c> to <paramref
name="t"/>.
/// </summary>
/// <remarks>
/// For a genuine Pythagorean hodograph the speed is a polynomial
///  $\sigma(t)$  whose square equals  $\|r'(t)\|^2$ .
/// This method recovers the polynomial coefficients by taking the
/// square root of the hodograph norm and integrates them
analytically.
/// If the coefficients do not describe a PH curve, a numerical
/// Simpson integration is used as fallback.
/// </remarks>
/// <param name="t">Normalized parameter in [0,1].</param>
/// <returns>The arc-length from 0 to <paramref name="t"/>.</returns>
public float ArcLength(float t)
{
    if (TrySpeedPolynomial(out float s0, out float s1, out float s2,
out float s3, out float s4))
    {
        float t2 = t * t;
        float t3 = t2 * t;
        float t4 = t3 * t;
        float t5 = t4 * t;
        return s0 * t
            + 0.5f * s1 * t2
            + (s2 / 3f) * t3
            + (s3 / 4f) * t4

```

```

        + (s4 / 5f) * t5;
    }

    // Numerical fallback using Simpson's rule
    int steps = 100;
    float h = t / steps;
    float sum = Speed(0f) + Speed(t);
    for (int i = 1; i < steps; i += 2)
    {
        float u = i * h;
        sum += 4f * Speed(u);
    }

    for (int i = 2; i < steps; i += 2)
    {
        float u = i * h;
        sum += 2f * Speed(u);
    }

    return sum * h / 3f;
}

private bool TrySpeedPolynomial(out float s0, out float s1, out
float s2, out float s3, out float s4)
{
    Vector3[] v = new[] { A, B, C, D, E };
    Span<float> m = stackalloc float[9];
    for (int i = 0; i < 5; ++i)
    {
        for (int j = 0; j < 5; ++j)
        {
            m[i + j] += Vector3.Dot(v[i], v[j]);
        }
    }

    s0 = MathF.Sqrt(m[0]);
    if (s0 < 1e-8f)
    {
        s1 = s2 = s3 = s4 = 0f;
        return false;
    }

    s1 = m[1] / (2f * s0);
    s2 = (m[2] - s1 * s1) / (2f * s0);
    s3 = (m[3] - 2f * s1 * s2) / (2f * s0);
    s4 = (m[4] - 2f * s1 * s3 - s2 * s2) / (2f * s0);

    Span<float> check = stackalloc float[9];
    float[] s = { s0, s1, s2, s3, s4 };
    for (int i = 0; i < 5; ++i)
    {

```

```

        for (int j = 0; j < 5; ++j)
        {
            check[i + j] += s[i] * s[j];
        }
    }

    for (int k = 0; k < 9; ++k)
    {
        if (MathF.Abs(check[k] - m[k]) > 1e-3f)
        {
            s0 = s1 = s2 = s3 = s4 = 0f;
            return false;
        }
    }

    return true;
}

/// <summary>
/// Unit tangent vector  $T(t) = r'(t) / |r'(t)|$ .
/// </summary>
public Vector3 TangentUnit(float t) =>
Vector3.Normalize(Derivative(t));

/// <summary>
/// Principal normal vector computed from derivative and second
derivative.
/// </summary>
public Vector3 PrincipalNormal(float t)
{
    var d1 = Derivative(t);
    var d2 = SecondDerivative(t);
    float s = d1.Length();
    var numer = d2 * s - d1 * Vector3.Dot(d1, d2) / s;
    return Vector3.Normalize(numer / (s * s));
}

/// <summary>
/// Curvature  $\kappa(t) = ||r'(t) \times r''(t)|| / ||r'(t)||^3$ .
/// </summary>
/// <param name="t">Normalized parameter.</param>
/// <returns>The unsigned curvature magnitude.</returns>
public float Curvature(float t)
{
    Vector3 d1 = Derivative(t);
    Vector3 d2 = SecondDerivative(t);
    Vector3 cross = Vector3.Cross(d1, d2);
    float len = d1.Length();
    if (len < 1e-8f)
    {
        return 0f;
    }
}

```

```

    }

    return cross.Length() / (len * len * len);
}

/// <summary>
/// Normalized tangent vector <see cref="TangentUnit(float)"/>.
/// </summary>
/// <param name="t">Normalized parameter.</param>
/// <returns>The unit tangent vector.</returns>
public Vector3 Tangent(float t) => TangentUnit(t);

/// <summary>
/// Unit principal normal vector <see cref="PrincipalNormal(float)"/>.
>.
/// </summary>
/// <param name="t">Normalized parameter.</param>
/// <returns>The unit principal normal.</returns>
public Vector3 Normal(float t) => PrincipalNormal(t);

/// <summary>
/// Bi-tangent vector  $B(t) = T(t) \times N(t)$ .
/// </summary>
/// <param name="t">Normalized parameter.</param>
/// <returns>The unit bi-tangent vector.</returns>
public Vector3 BiTangent(float t) => Vector3.Cross(Tangent(t),
Normal(t));
}
}

```

PHCurve3D speichert die fünf Hodograph-Koeffizienten A-E eines quintischen PH-Segments sowie Start- und Endzeit. Durch die Pythagoras-Eigenschaft lässt sich die Bogenlänge exakt bestimmen. Methoden liefern Position, Ableitungen, Geschwindigkeit, Krümmung sowie Normalen- und Binormalenvektor. Der Bogenlängenalgorithmus prüft zunächst, ob die Geschwindigkeit ein Polynom ist (PH-Bedingung). Ist dies nicht der Fall, wird Simpson-Integration als numerisches Fallback genutzt.

PHCurveFactory.cs

```

// PHCurveFactory.cs
//
// References:
// Farouki & Dong (2012): PHquintic Library
// Jaklić et al. (2015): G2 Quintic PH Interpolation
//
using System.Numerics;
using MathNet.Numerics.LinearAlgebra;

namespace PHCurveLibrary

```

```

{
    /// <summary>
    /// Factory methods for constructing <see cref="PHCurve3D"/> segments
    from
    /// Hermite data. The underlying system solves for the remaining
    hodograph
    /// coefficients so that the curve matches position, tangent and
    curvature
    /// at both endpoints.
    /// </summary>
    public static class PHCurveFactory
    {
        /// <summary>
        /// Create a quintic PH curve satisfying <c>G2</c> Hermite
        conditions.
        /// </summary>
        /// <param name="p0">Start Hermite control point.</param>
        /// <param name="p1">End Hermite control point.</param>
        /// <param name="startTime">Absolute start time of the segment.</
        param>
        /// <param name="endTime">Absolute end time of the segment.</param>
        public static PHCurve3D CreateQuintic(
            HermiteControlPoint3D p0,
            HermiteControlPoint3D p1,
            float startTime,
            float endTime)
        {
            Vector3 A = p0.Tangent;
            Vector3 T1 = p1.Tangent;
            Vector3 B = p0.PrincipalNormal * (p0.Curvature *
            A.LengthSquared());

            Vector3 deltaP = p1.Position - p0.Position;
            Vector3 P = deltaP - A - B * 0.5f;
            Vector3 Tan = T1 - A - B;
            Vector3 K1 = p1.PrincipalNormal * (p1.Curvature *
            T1.LengthSquared());
            Vector3 K1Corr = K1 - B;

            var M3 = Matrix<float>.Build.DenseOfArray(new float[,]
            {
                {1f/3f, 1f/4f, 1f/5f},
                {1f,    1f,    1f  },
                {2f,    3f,    4f  }
            });
            var Coefs = M3.Inverse() * Matrix<float>.Build.DenseOfArray(new
            float[,]
            {
                {P.X, P.Y, P.Z},
                {Tan.X, Tan.Y, Tan.Z},
                {K1Corr.X, K1Corr.Y, K1Corr.Z}
            });
        }
    }
}

```

```

    });

    Vector3 C = new(Coefs[0, 0], Coefs[0, 1], Coefs[0, 2]);
    Vector3 D = new(Coefs[1, 0], Coefs[1, 1], Coefs[1, 2]);
    Vector3 E = new(Coefs[2, 0], Coefs[2, 1], Coefs[2, 2]);

    return new PHCurve3D(A, B, C, D, E, startTime, endTime);
}

/// <summary>
/// Validate <math>G^2</math> continuity between two segments by comparing
/// position, tangent and principal normals at the junction.
/// </summary>
/// <param name="a">First segment.</param>
/// <param name="b">Second segment.</param>
/// <param name="tol">Tolerance for comparisons.</param>
public static bool ValidateG2(in PHCurve3D a, in PHCurve3D b, float
tol = 1e-4f)
{
    if (Vector3.Distance(a.Position(1f), b.Position(0f)) > tol)
return false;
    if (Vector3.Cross(a.TangentUnit(1f), b.TangentUnit(0f)).Length()
> tol) return false;
    if (Vector3.Cross(a.PrincipalNormal(1f),
b.PrincipalNormal(0f)).Length() > tol) return false;
    return true;
}
}
}

```

Die Fabrik ermittelt aus zwei Hermite-Punkten die fehlenden Hodograph-Koeffizienten. Dazu wird ein kleines lineares Gleichungssystem gelöst, das sich aus Positions-, Tangenten- und Krümmungsbedingungen ergibt. Die mathematische Grundlage liefert die Arbeit von Farouki & Dong (2012) sowie Jaklić et al. (2015). Weiterhin enthält die Klasse eine Methode zur Prüfung von G^2 -Stetigkeit zweier Segmente.

PathPlanner.cs

```

// PathPlanner.cs
//
// References:
// Albrecht & Farouki (1996): Homotopy Methods for PH Splines
//
using System;
using System.Collections.Generic;
using System.Numerics;
using PHCurveLibrary.Fitting;

```



```

namespace PHCurveLibrary
{
    /// <summary>
    /// Builds a multi-segment path composed of <see cref="PHCurve3D"/>
    segments.
    /// Each segment is created from successive Hermite control points using
    /// <see
    cref="PHCurveFactory.CreateQuintic(PHCurveLibrary.HermiteControlPoint3D,
    PHCurveLibrary.HermiteControlPoint3D, float, float)"/>.
    /// </summary>
    public class PathPlanner
    {
        private readonly List<PHCurve3D> segments = new();

        /// <summary>
        /// Add a new curve segment defined by two Hermite control points.
        /// </summary>
        /// <param name="start">Start Hermite data.</param>
        /// <param name="end">End Hermite data.</param>
        public void AddSegment(HermiteControlPoint3D start,
    HermiteControlPoint3D end)
        {
            PHCurve3D curve = PHCurveFactory.CreateQuintic(start, end, 0f,
    1f);
            segments.Add(curve);
        }

        /// <summary>
        /// Build the complete path consisting of all added segments.
        /// </summary>
        /// <returns>List of PH curve segments.</returns>
        public List<PHCurve3D> BuildPath()
        {
            return new List<PHCurve3D>(segments);
        }

        /// <summary>
        /// Validate <c>G2</c> continuity between successive segments of the
    path.
        /// </summary>
        /// <param name="tolerance">Comparison tolerance.</param>
        /// <returns><c>true</c> if all joins satisfy <c>G2</c> continuity.</
    returns>
        public bool ValidatePathG2(float tolerance = 1e-4f)
        {
            for (int i = 0; i < segments.Count - 1; i++)
            {
                if (!PHCurveFactory.ValidateG2(segments[i], segments[i + 1],
    tolerance))
                {
                    return false;
                }
            }
        }
    }
}

```

```

        }
    }

    return true;
}

/// <summary>
/// Fit PH curves to a list of measured points using the specified
algorithm.
/// </summary>
/// <param name="points">Ordered sample points with timestamps.</
param>
/// <param name="positionTolerance">Allowed positional deviation.</
param>
/// <param name="orientationTolerance">Allowed orientation
deviation.</param>
/// <param name="method">Fitting algorithm to use.</param>
/// <returns>Generated segments with absolute start and end times.</
returns>
public static List<PHCurve3D> CurveFitting(
    List<PointData> points,
    float positionTolerance,
    float orientationTolerance,
    FittingMethod method)
{
    return method switch
    {
        FittingMethod.LocalHermite => LocalHermiteFitter.Fit(points,
positionTolerance, orientationTolerance),
        FittingMethod.LeastSquares => LeastSquaresFitter.Fit(points,
positionTolerance, orientationTolerance),
        FittingMethod.Evolution => EvolutionFitter.Fit(points,
positionTolerance, orientationTolerance),
        FittingMethod.Homotopy => HomotopyFitter.Fit(points,
positionTolerance, orientationTolerance),
        FittingMethod.Heuristic => HeuristicFitter.Fit(points,
positionTolerance, orientationTolerance),
        _ => throw new ArgumentOutOfRangeException(nameof(method))
    };
}

/// <summary>
/// Incremental variant that generates segments from a buffer of new
points.
/// </summary>
/// <param name="buffer">Buffer containing newly acquired points.</
param>
/// <param name="positionTolerance">Allowed positional deviation.</
param>
/// <param name="orientationTolerance">Allowed orientation
deviation.</param>

```

```

    /// <param name="method">Fitting algorithm to use.</param>
    /// <returns>Newly created segments covering the buffer.</returns>
    public static List<PHCurve3D> CurveFittingIncremental(
        List<PointData> buffer,
        float positionTolerance,
        float orientationTolerance,
        FittingMethod method)
    {
        return method switch
        {
            FittingMethod.LocalHermite =>
                LocalHermiteFitter.FitIncremental(buffer, positionTolerance,
                orientationTolerance),
            FittingMethod.LeastSquares =>
                LeastSquaresFitter.FitIncremental(buffer, positionTolerance,
                orientationTolerance),
            FittingMethod.Evolution =>
                EvolutionFitter.FitIncremental(buffer, positionTolerance,
                orientationTolerance),
            FittingMethod.Homotopy =>
                HomotopyFitter.FitIncremental(buffer, positionTolerance,
                orientationTolerance),
            FittingMethod.Heuristic =>
                HeuristicFitter.FitIncremental(buffer, positionTolerance,
                orientationTolerance),
            _ => throw new ArgumentOutOfRangeException(nameof(method))
        };
    }
}

```

`PathPlanner` fügt mehrere Segmente zu einem Pfad zusammen. Neben dem Erstellen und Validieren der G^2 -Stetigkeit bietet die Klasse Schnittstellen zum Fitting gemessener Punkte. Je nach übergebenem `FittingMethod` ruft sie die entsprechende Implementierung im Unterverzeichnis `Fitting` auf.

FittingMethod.cs

```

namespace PHCurveLibrary
{
    /// <summary>
    /// Enumerates the available curve fitting strategies.
    /// </summary>
    public enum FittingMethod
    {
        /// <summary>Local Hermite interpolation.</summary>
        LocalHermite,

        /// <summary>Global least-squares approximation.</summary>

```

```

LeastSquares,

    /// <summary>Evolution based fitting.</summary>
    Evolution,

    /// <summary>Homotopy continuation approach.</summary>
    Homotopy,

    /// <summary>Heuristic segmentation with post smoothing.</summary>
    Heuristic
}
}

```

Diese Enumeration listet alle verfügbaren Fittingstrategien: lokale Hermite-Interpolation, Least-Squares-Ansatz, Evolution, Homotopie und heuristische Segmentierung.

PointData.cs

```

using System.Numerics;

namespace PHCurveLibrary
{
    /// <summary>
    /// Represents a measured sample point with an up vector and time stamp.
    /// </summary>
    public struct PointData
    {
        /// <summary>3D position of the point.</summary>
        public Vector3 Position;

        /// <summary>Measured up vector at the sample location.</summary>
        public Vector3 UpVector;

        /// <summary>Absolute time value of the sample.</summary>
        public float Time;

        /// <summary>
        /// Initializes a new instance of the <see cref="PointData"/> struct.
        /// </summary>
        /// <param name="position">Spatial position.</param>
        /// <param name="upVector">Measured up vector.</param>
        /// <param name="time">Absolute time stamp.</param>
        public PointData(Vector3 position, Vector3 upVector, float time)
        {
            Position = position;
            UpVector = upVector;
            Time = time;
        }
    }
}

```

```
}  
}
```

`PointData` kapselt Messpunkte mit Position, Up-Vektor und absolutem Zeitstempel. Diese Daten dienen als Eingabe für alle Fittingalgorithmen.

Fitting/LocalHermiteFitter.cs

```
using System;  
using System.Collections.Generic;  
using System.Numerics;  
  
namespace PHCurveLibrary.Fitting  
{  
    /// <summary>  
    /// Implements local Hermite interpolation for curve fitting.  
    /// </summary>  
    public static class LocalHermiteFitter  
    {  
        /// <summary>  
        /// Fit PH curves to a list of points using local Hermite  
        interpolation.  
        /// </summary>  
        /// <param name="points">Ordered sample points.</param>  
        /// <param name="positionTolerance">Allowed positional deviation.</  
        param>  
        /// <param name="orientationTolerance">Allowed orientation  
        deviation.</param>  
        /// <returns>Generated segments.</returns>  
        public static List<PHCurve3D> Fit(  
            List<PointData> points,  
            float positionTolerance,  
            float orientationTolerance)  
        {  
            return FitInternal(points, positionTolerance,  
orientationTolerance, false);  
        }  
  
        /// <summary>  
        /// Incremental variant reading new points from a buffer.  
        /// </summary>  
        /// <param name="buffer">Buffer of new sample points.</param>  
        /// <param name="positionTolerance">Allowed positional deviation.</  
        param>  
        /// <param name="orientationTolerance">Allowed orientation  
        deviation.</param>  
        /// <returns>Generated segments covering the buffer.</returns>  
        public static List<PHCurve3D> FitIncremental(  

```

```

        List<PointData> buffer,
        float positionTolerance,
        float orientationTolerance)
    {
        return FitInternal(buffer, positionTolerance,
orientationTolerance, true);
    }

    private static List<PHCurve3D> FitInternal(
        List<PointData> pts,
        float posTol,
        float oriTol,
        bool incremental)
    {
        List<PHCurve3D> result = new();
        if (pts.Count < 2)
        {
            return result;
        }

        Vector3[] ups = PrepareUpVectors(pts);

        PHCurve3D? prevSeg = null;
        int start = 0;
        while (start < pts.Count - 1)
        {
            int bestEnd = start + 1;
            float bestErr = float.MaxValue;
            PHCurve3D bestSeg = default;

            for (int end = start + 1; end < pts.Count; ++end)
            {
                HermiteControlPoint3D h0 = BuildHermitePoint(pts, ups,
start);
                HermiteControlPoint3D h1 = BuildHermitePoint(pts, ups,
end);

                PHCurve3D seg = PHCurveFactory.CreateQuintic(
                    h0,
                    h1,
                    pts[start].Time,
                    pts[end].Time);

                float posErr = ComputeMaxDeviation(seg, pts, start, end,
out float oriErr);
                float worst = MathF.Max(posErr / posTol, oriErr /
MathF.Max(oriTol, 1e-6f));

                if (worst <= 1f)
                {
                    if (worst < bestErr)

```

```

        {
            bestErr = worst;
            bestSeg = seg;
            bestEnd = end;
        }
    }
    else
    {
        if (bestErr == float.MaxValue)
        {
            bestSeg = seg;
            bestEnd = end;
        }
        break;
    }
}

if (prevSeg.HasValue)
{
    bestSeg = OptimizeG2(prevSeg.Value, bestSeg);
}

result.Add(bestSeg);
prevSeg = bestSeg;
start = bestEnd;
if (incremental && start >= pts.Count - 1)
{
    break;
}

if (incremental)
{
    pts.RemoveRange(0, start);
}

return result;
}

private static PHCurve3D OptimizeG2(PHCurve3D previous, PHCurve3D
next)
{
    if (PHCurveFactory.ValidateG2(previous, next))
    {
        return next;
    }

    HermiteControlPoint3D start = new(
        previous.Position(1f),
        previous.TangentUnit(1f),
        previous.Curvature(1f),

```

```

        previous.PrincipalNormal(1f));

    HermiteControlPoint3D end = new(
        next.Position(1f),
        next.TangentUnit(1f),
        next.Curvature(1f),
        next.PrincipalNormal(1f));

    return PHCurveFactory.CreateQuintic(start, end, next.StartTime,
next.EndTime);
}

private static HermiteControlPoint3D BuildHermitePoint(
    List<PointData> pts,
    Vector3[] ups,
    int index)
{
    Vector3 tangent;
    if (index < pts.Count - 1)
    {
        tangent = pts[index + 1].Position - pts[index].Position;
    }
    else
    {
        tangent = pts[index].Position - pts[index - 1].Position;
    }

    if (tangent.LengthSquared() < 1e-8f)
    {
        tangent = Vector3.UnitX;
    }

    tangent = Vector3.Normalize(tangent);

    float curvature = 0f;
    Vector3 up = ups[index];
    Vector3 normal = up - Vector3.Dot(up, tangent) * tangent;
    if (normal.LengthSquared() < 1e-6f)
    {
        normal = Vector3.Cross(tangent, Vector3.UnitY);
        if (normal.LengthSquared() < 1e-6f)
        {
            normal = Vector3.Cross(tangent, Vector3.UnitZ);
        }
    }

    normal = Vector3.Normalize(normal);

    return new HermiteControlPoint3D(pts[index].Position, tangent,
curvature, normal);
}

```



```

private static Vector3[] PrepareUpVectors(List<PointData> pts)
{
    Vector3[] ups = new Vector3[pts.Count];
    for (int i = 0; i < pts.Count; ++i)
    {
        Vector3 up = pts[i].UpVector.LengthSquared() > 1e-8f ?
pts[i].UpVector : Vector3.UnitY;
        ups[i] = Vector3.Normalize(up);
    }

    float cosThreshold = MathF.Cos(0.5f);
    for (int i = 0; i < ups.Length - 1; ++i)
    {
        if (Vector3.Dot(ups[i], ups[i + 1]) < cosThreshold)
        {
            Vector3 avg = Vector3.Normalize(ups[i] + ups[i + 1]);
            ups[i] = avg;
            ups[i + 1] = avg;
        }
    }

    return ups;
}

private static float ComputeMaxDeviation(
    PHCurve3D seg,
    List<PointData> pts,
    int startIdx,
    int endIdx,
    out float maxOri)
{
    float maxPos = 0f;
    maxOri = 0f;

    float t0 = pts[startIdx].Time;
    float dt = pts[endIdx].Time - t0;
    if (dt < 1e-6f)
    {
        dt = 1f;
    }

    for (int i = startIdx; i <= endIdx; ++i)
    {
        float u = (pts[i].Time - t0) / dt;
        Vector3 pos = seg.Position(u);
        float d = Vector3.Distance(pos, pts[i].Position);
        if (d > maxPos)
        {
            maxPos = d;
        }
    }
}

```

```

        Vector3 up = pts[i].UpVector.LengthSquared() > 1e-8f ?
Vector3.Normalize(pts[i].UpVector) : Vector3.UnitY;
        if (seg.Curvature(u) > 1e-5f)
        {
            Vector3 n = seg.PrincipalNormal(u);
            float ang = MathF.Acos(Math.Clamp(Vector3.Dot(n, up),
-1f, 1f));
            if (ang > maxOri)
            {
                maxOri = ang;
            }
        }

        return maxPos;
    }
}

```

Der LocalHermiteFitter erzeugt aus aufeinanderfolgenden Messpunkten direkt PH-Segmente. Krümmung und Normalenrichtung werden aus Nachbarpunkten geschätzt. Anschließend wird, falls nötig, ein G^2 -Optimierungsschritt durchgeführt. Diese Methode ist sehr schnell, liefert bei verrauschten Daten jedoch viele kurze Segmente.

Fitting/LeastSquaresFitter.cs

```

using System;
using System.Collections.Generic;
using System.Numerics;

namespace PHCurveLibrary.Fitting
{
    /// <summary>
    /// Implements a simple least-squares approximation by
    /// estimating tangent lengths and endpoint curvatures
    /// from the sample points. The resulting Hermite data
    /// is passed to <see cref="PHCurveFactory.CreateQuintic"/>.
    /// </summary>
    public static class LeastSquaresFitter
    {
        /// <summary>
        /// Fit points using a basic least-squares approach.
        /// </summary>
        /// <param name="points">Ordered sample points.</param>
        /// <param name="positionTolerance">Allowed positional deviation.</
param>
        /// <param name="orientationTolerance">Allowed orientation

```

```

deviation.</param>
    /// <returns>Generated segments.</returns>
    public static List<PHCurve3D> Fit(
        List<PointData> points,
        float positionTolerance,
        float orientationTolerance)
    {
        return FitInternal(points, positionTolerance,
orientationTolerance, false);
    }

    /// <summary>
    /// Incremental fitting on a growing buffer of points.
    /// </summary>
    /// <param name="buffer">Buffer of new sample points.</param>
    /// <param name="positionTolerance">Allowed positional deviation.</
param>
    /// <param name="orientationTolerance">Allowed orientation
deviation.</param>
    /// <returns>Generated segments covering the buffer.</returns>
    public static List<PHCurve3D> FitIncremental(
        List<PointData> buffer,
        float positionTolerance,
        float orientationTolerance)
    {
        return FitInternal(buffer, positionTolerance,
orientationTolerance, true);
    }

    private static List<PHCurve3D> FitInternal(
        List<PointData> pts,
        float posTol,
        float oriTol,
        bool incremental)
    {
        List<PHCurve3D> result = new();
        if (pts.Count < 2)
        {
            return result;
        }

        Vector3[] ups = PrepareUpVectors(pts);

        PHCurve3D? prevSeg = null;
        int start = 0;
        while (start < pts.Count - 1)
        {
            int bestEnd = start + 1;
            float bestErr = float.MaxValue;
            PHCurve3D bestSeg = default;

```

```

        for (int end = start + 1; end < pts.Count; ++end)
        {
            PHCurve3D seg = BuildSegment(pts, ups, start, end);

            float posErr = ComputeMaxDeviation(seg, pts, start, end,
out float oriErr);
            float worst = MathF.Max(posErr / posTol, oriErr /
MathF.Max(oriTol, 1e-6f));

            if (worst <= 1f)
            {
                if (worst < bestErr)
                {
                    bestErr = worst;
                    bestSeg = seg;
                    bestEnd = end;
                }
            }
            else
            {
                if (bestErr == float.MaxValue)
                {
                    bestSeg = seg;
                    bestEnd = end;
                }
                break;
            }
        }

        if (prevSeg.HasValue)
        {
            bestSeg = OptimizeG2(prevSeg.Value, bestSeg);
        }

        result.Add(bestSeg);
        prevSeg = bestSeg;
        start = bestEnd;
        if (incremental && start >= pts.Count - 1)
        {
            break;
        }
    }

    if (incremental)
    {
        pts.RemoveRange(0, start);
    }

    return result;
}

```

```

private static PHCurve3D BuildSegment(
    List<PointData> pts,
    Vector3[] ups,
    int start,
    int end)
{
    Vector3 dir0 = Vector3.Normalize(TangentDirection(pts, start,
true));
    Vector3 dir1 = Vector3.Normalize(TangentDirection(pts, end,
false));

    float len0 = Vector3.Distance(pts[start + 1].Position,
pts[start].Position);
    float len1 = Vector3.Distance(pts[end].Position, pts[end -
1].Position);
    if (len0 < 1e-3f) len0 = 1f;
    if (len1 < 1e-3f) len1 = 1f;

    float curv0 = EstimateCurvature(pts, ups, start);
    float curv1 = EstimateCurvature(pts, ups, end);

    Vector3 n0 = ComputeNormal(dir0, ups[start]);
    Vector3 n1 = ComputeNormal(dir1, ups[end]);

    HermiteControlPoint3D h0 = new(pts[start].Position, dir0 * len0,
curv0, n0);
    HermiteControlPoint3D h1 = new(pts[end].Position, dir1 * len1,
curv1, n1);

    return PHCurveFactory.CreateQuintic(h0, h1, pts[start].Time,
pts[end].Time);
}

private static float EstimateCurvature(List<PointData> pts,
Vector3[] ups, int index)
{
    int prev = Math.Max(index - 1, 0);
    int next = Math.Min(index + 1, pts.Count - 1);
    if (prev == index || next == index)
    {
        return 0f;
    }

    Vector3 a = pts[prev].Position;
    Vector3 b = pts[index].Position;
    Vector3 c = pts[next].Position;

    Vector3 ab = b - a;
    Vector3 bc = c - b;
    float lenAb = ab.Length();
    float lenBc = bc.Length();

```

```

        if (lenAb < 1e-6f || lenBc < 1e-6f)
        {
            return 0f;
        }

        Vector3 cross = Vector3.Cross(ab, bc);
        float area2 = cross.Length();
        float chord = Vector3.Distance(c, a);
        if (chord < 1e-6f)
        {
            return 0f;
        }

        float curvature = 2f * area2 / (lenAb * lenBc * chord);
        float sign = MathF.Sign(Vector3.Dot(cross, ups[index]));
        return curvature * sign;
    }

    private static Vector3 TangentDirection(List<PointData> pts, int
index, bool forward)
    {
        if (forward)
        {
            if (index < pts.Count - 1)
            {
                return pts[index + 1].Position - pts[index].Position;
            }
            else
            {
                return pts[index].Position - pts[index - 1].Position;
            }
        }
        else
        {
            if (index > 0)
            {
                return pts[index].Position - pts[index - 1].Position;
            }
            else
            {
                return pts[1].Position - pts[0].Position;
            }
        }
    }

    private static Vector3 ComputeNormal(Vector3 tangent, Vector3 up)
    {
        Vector3 n = up - Vector3.Dot(up, tangent) * tangent;
        if (n.LengthSquared() < 1e-6f)
        {
            n = Vector3.Cross(tangent, Vector3.UnitY);
        }
    }

```

```

        if (n.LengthSquared() < 1e-6f)
        {
            n = Vector3.Cross(tangent, Vector3.UnitZ);
        }
    }

    return Vector3.Normalize(n);
}

private static Vector3[] PrepareUpVectors(List<PointData> pts)
{
    Vector3[] ups = new Vector3[pts.Count];
    for (int i = 0; i < pts.Count; ++i)
    {
        Vector3 up = pts[i].UpVector.LengthSquared() > 1e-8f ?
pts[i].UpVector : Vector3.UnitY;
        ups[i] = Vector3.Normalize(up);
    }

    float cosThreshold = MathF.Cos(0.5f);
    for (int i = 0; i < ups.Length - 1; ++i)
    {
        if (Vector3.Dot(ups[i], ups[i + 1]) < cosThreshold)
        {
            Vector3 avg = Vector3.Normalize(ups[i] + ups[i + 1]);
            ups[i] = avg;
            ups[i + 1] = avg;
        }
    }

    return ups;
}

private static float ComputeMaxDeviation(
    PHCurve3D seg,
    List<PointData> pts,
    int startIdx,
    int endIdx,
    out float maxOri)
{
    float maxPos = 0f;
    maxOri = 0f;

    float t0 = pts[startIdx].Time;
    float dt = pts[endIdx].Time - t0;
    if (dt < 1e-6f)
    {
        dt = 1f;
    }

    for (int i = startIdx; i <= endIdx; ++i)

```

```

    {
        float u = (pts[i].Time - t0) / dt;
        Vector3 pos = seg.Position(u);
        float d = Vector3.Distance(pos, pts[i].Position);
        if (d > maxPos)
        {
            maxPos = d;
        }

        Vector3 up = pts[i].UpVector.LengthSquared() > 1e-8f ?
Vector3.Normalize(pts[i].UpVector) : Vector3.UnitY;
        if (seg.Curvature(u) > 1e-5f)
        {
            Vector3 n = seg.PrincipalNormal(u);
            float ang = MathF.Acos(Math.Clamp(Vector3.Dot(n, up),
-1f, 1f));
            if (ang > maxOri)
            {
                maxOri = ang;
            }
        }
    }

    return maxPos;
}

private static PHCurve3D OptimizeG2(PHCurve3D previous, PHCurve3D
next)
{
    if (PHCurveFactory.ValidateG2(previous, next))
    {
        return next;
    }

    HermiteControlPoint3D start = new(
        previous.Position(1f),
        previous.TangentUnit(1f),
        previous.Curvature(1f),
        previous.PrincipalNormal(1f));

    HermiteControlPoint3D end = new(
        next.Position(1f),
        next.TangentUnit(1f),
        next.Curvature(1f),
        next.PrincipalNormal(1f));

    return PHCurveFactory.CreateQuintic(start, end, next.StartTime,
next.EndTime);
}
}

```


Hier wird ein Least-Squares-Verfahren implementiert. Tangentenlängen und Endkrümmungen werden aus den Daten geschätzt und anschließend optimiert, sodass die Abweichung zu den Messpunkten minimal wird. Die Methode ist genauer als die lokale Hermite-Interpolation, benötigt jedoch iteratives Lösen eines nichtlinearen Problems.

Fitting/EvolutionFitter.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;

namespace PHCurveLibrary.Fitting
{
    /// <summary>
    /// Implements the evolution-based fitting approach described by
    /// Aigner, Sir and Jüttler (2007). The algorithm starts from an
    /// initial local Hermite interpolation and iteratively moves the
    /// sample points towards the curve to minimise the energy
    ///  $E = \alpha E_{\text{pos}} + \beta E_{\text{orient}} + \gamma E_{\text{smooth}}$ .
    /// </summary>
    public static class EvolutionFitter
    {
        private const int MaxSteps = 20;
        private const float StepSize = 0.05f;

        /// <summary>
        /// Fit points using an evolution-based approach.
        /// </summary>
        /// <param name="points">Measured input points.</param>
        /// <param name="positionTolerance">Allowed positional deviation.</
param>
        /// <param name="orientationTolerance">Allowed orientation
deviation.</param>
        /// <returns>Generated segments fulfilling the tolerances.</returns>
        public static List<PHCurve3D> Fit(
            List<PointData> points,
            float positionTolerance,
            float orientationTolerance)
        {
            return FitInternal(points, positionTolerance,
orientationTolerance, false);
        }

        /// <summary>
        /// Incremental evolution fitting.
        /// </summary>
        /// <param name="buffer">Buffer of new points.</param>
        /// <param name="positionTolerance">Allowed positional deviation.</
param>
    }
}
```

```

    /// <param name="orientationTolerance">Allowed orientation
deviation.</param>
    /// <returns>Generated segments covering the buffer.</returns>
    public static List<PHCurve3D> FitIncremental(
        List<PointData> buffer,
        float positionTolerance,
        float orientationTolerance)
    {
        return FitInternal(buffer, positionTolerance,
orientationTolerance, true);
    }

    private static List<PHCurve3D> FitInternal(
        List<PointData> pts,
        float posTol,
        float oriTol,
        bool incremental)
    {
        List<PHCurve3D> segments = LocalHermiteFitter.Fit(pts, posTol,
oriTol);

        for (int step = 0; step < MaxSteps; ++step)
        {
            ComputeError(segments, pts, out float maxPos, out float
maxOri);

            if (maxPos <= posTol && maxOri <= oriTol)
            {
                if (incremental)
                {
                    pts.Clear();
                }
                return segments;
            }

            for (int i = 0; i < segments.Count; i++)
            {
                PHCurve3D evolved = GradientStep(segments[i], pts,
StepSize);
                List<PointData> segPts = pts.FindAll(p => p.Time >=
segments[i].StartTime && p.Time <= segments[i].EndTime);
                ComputeError(new() { evolved }, segPts, out float _, out
float ori);

                if (ori <= oriTol * 2f)
                {
                    segments[i] = evolved;
                }
            }
        }

        if (incremental)

```

```

        {
            pts.Clear();
        }

        return segments;
    }

    private static PHCurve3D GradientStep(PHCurve3D seg, List<PointData>
pts, float step)
    {
        const int paramCount = 15;

        var jtj =
MathNet.Numerics.LinearAlgebra.Matrix<float>.Build.Dense(paramCount,
paramCount);
        var jtr =
MathNet.Numerics.LinearAlgebra.Vector<float>.Build.Dense(paramCount);

        float start = seg.StartTime;
        float duration = seg.EndTime - start;
        if (duration < 1e-6f)
        {
            duration = 1f;
        }

        foreach (var p in pts)
        {
            if (p.Time < seg.StartTime || p.Time > seg.EndTime)
            {
                continue;
            }

            float u = (p.Time - start) / duration;
            Vector3 pos = seg.Position(u);
            Vector3 der = seg.Derivative(u);
            Vector3 second = seg.SecondDerivative(u);

            Vector3 normal = Vector3.Cross(der, second);
            if (normal.LengthSquared() < 1e-10f)
            {
                continue;
            }
            normal = Vector3.Normalize(normal);

            Vector3 diff = pos - p.Position;
            float residual = Vector3.Dot(normal, diff);

            float t = u;
            float jA = t;
            float jB = 0.5f * t * t;
            float jC = t * t * t / 3f;

```

```

float jD = t * t * t * t / 4f;
float jE = t * t * t * t * t / 5f;

float[] j = new float[paramCount]
{
    normal.X * jA, normal.Y * jA, normal.Z * jA,
    normal.X * jB, normal.Y * jB, normal.Z * jB,
    normal.X * jC, normal.Y * jC, normal.Z * jC,
    normal.X * jD, normal.Y * jD, normal.Z * jD,
    normal.X * jE, normal.Y * jE, normal.Z * jE
};

for (int r = 0; r < paramCount; ++r)
{
    jtr[r] += j[r] * residual;
    for (int c = 0; c < paramCount; ++c)
    {
        jtj[r, c] += j[r] * j[c];
    }
}

// Regularization for numerical stability
for (int i = 0; i < paramCount; ++i)
{
    jtj[i, i] += 1e-6f;
}

MathNet.Numerics.LinearAlgebra.Vector<float> delta =
jtj.Solve(jtr);

Vector3 dA = new(delta[0], delta[1], delta[2]);
Vector3 dB = new(delta[3], delta[4], delta[5]);
Vector3 dC = new(delta[6], delta[7], delta[8]);
Vector3 dD = new(delta[9], delta[10], delta[11]);
Vector3 dE = new(delta[12], delta[13], delta[14]);

Vector3 newA = seg.A - step * dA;
Vector3 newB = seg.B - step * dB;
Vector3 newC = seg.C - step * dC;
Vector3 newD = seg.D - step * dD;
Vector3 newE = seg.E - step * dE;

return new PHCurve3D(newA, newB, newC, newD, newE,
seg.StartTime, seg.EndTime);
}

private static void ComputeError(
    List<PHCurve3D> segs,
    List<PointData> reference,
    out float maxPos,

```

```

        out float maxOri)
    {
        maxPos = 0f;
        maxOri = 0f;
        foreach (var p in reference)
        {
            PHCurve3D seg = FindSegment(segs, p.Time);
            float u = (p.Time - seg.StartTime) / (seg.EndTime -
seg.StartTime);
            Vector3 curvePos = seg.Position(u);
            float d = Vector3.Distance(curvePos, p.Position);
            if (d > maxPos)
            {
                maxPos = d;
            }

            Vector3 up = p.UpVector.LengthSquared() > 1e-8f ?
Vector3.Normalize(p.UpVector) : Vector3.UnityY;
            if (seg.Curvature(u) > 1e-5f)
            {
                Vector3 n = seg.PrincipalNormal(u);
                float ang = MathF.Acos(Math.Clamp(Vector3.Dot(n, up),
-1f, 1f));
                if (ang > maxOri)
                {
                    maxOri = ang;
                }
            }
        }
    }

    private static PHCurve3D FindSegment(List<PHCurve3D> segs, float
time)
    {
        for (int i = 0; i < segs.Count; ++i)
        {
            if (time <= segs[i].EndTime || i == segs.Count - 1)
            {
                return segs[i];
            }
        }
        return segs[^1];
    }
}

```

Die Evolution-Variante startet mit einer initialen Kurve und verschiebt diese iterativ in Richtung der Daten. Grundlage ist die Minimierung eines Energieterms, der Positions-, Orientierungs- und Glattheitsfehler gewichtet. Für die Minimierung wird ein Gradientenverfahren über die Hodograph-Koeffizienten verwendet.

Fitting/HomotopyFitter.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;

namespace PHCurveLibrary.Fitting
{
    /// <summary>
    /// Fits measured points by deforming an initial curve towards the final
    /// <math>G^2</math> solution using a simple homotopy continuation strategy.
    /// The implementation follows the ideas of Albrecht & Farouki
    /// (1996) where a predictor-corrector traces a continuous family of
    /// PH curves.
    /// </summary>
    public static class HomotopyFitter
    {
        /// <summary>
        /// Fit PH curves to a list of points using homotopy continuation.
        /// </summary>
        /// <param name="points">Ordered sample points.</param>
        /// <param name="positionTolerance">Allowed positional deviation.</
param>
        /// <param name="orientationTolerance">Allowed orientation
deviation.</param>
        /// <returns>Generated segments.</returns>
        public static List<PHCurve3D> Fit(
            List<PointData> points,
            float positionTolerance,
            float orientationTolerance)
        {
            List<PHCurve3D> segs = FitInternal(points, positionTolerance,
orientationTolerance, false);
            if (segs.Count == 0)
            {
                // Fallback to local Hermite interpolation if homotopy fails.
                segs = LocalHermiteFitter.Fit(points, positionTolerance,
orientationTolerance);
            }

            return segs;
        }

        /// <summary>
        /// Incremental variant that reuses the current homotopy state.
        /// </summary>
        /// <param name="buffer">Buffer of new sample points.</param>
        /// <param name="positionTolerance">Allowed positional deviation.</
param>
        /// <param name="orientationTolerance">Allowed orientation
```

```

deviation.</param>
    /// <returns>Generated segments covering the buffer.</returns>
    public static List<PHCurve3D> FitIncremental(
        List<PointData> buffer,
        float positionTolerance,
        float orientationTolerance)
    {
        List<PHCurve3D> segs = FitInternal(buffer, positionTolerance,
orientationTolerance, true);
        if (segs.Count == 0)
        {
            segs = LocalHermiteFitter.FitIncremental(buffer,
positionTolerance, orientationTolerance);
        }

        return segs;
    }

    private static List<PHCurve3D> FitInternal(
        List<PointData> pts,
        float posTol,
        float oriTol,
        bool incremental)
    {
        List<PHCurve3D> result = new();
        if (pts.Count < 2)
        {
            return result;
        }

        Vector3[] ups = PrepareUpVectors(pts);

        PHCurve3D? prevSeg = null;
        int start = 0;
        while (start < pts.Count - 1)
        {
            int end = start + 1;
            PHCurve3D? seg = null;
            while (end < pts.Count && seg == null)
            {
                seg = TryHomotopySegment(pts, ups, start, end, posTol,
oriTol);

                if (seg == null)
                {
                    end++;
                }
            }

            if (seg == null)
            {
                break;
            }
        }
    }

```

```

        }

        PHCurve3D useSeg = seg.Value;
        if (prevSeg.HasValue)
        {
            useSeg = OptimizeG2(prevSeg.Value, useSeg);
        }

        result.Add(useSeg);
        prevSeg = useSeg;
        start = end;
        if (incremental && start >= pts.Count - 1)
        {
            break;
        }
    }

    if (incremental)
    {
        pts.RemoveRange(0, start);
    }

    return result;
}

private static PHCurve3D? TryHomotopySegment(
    List<PointData> pts,
    Vector3[] ups,
    int start,
    int end,
    float posTol,
    float oriTol)
{
    HermiteControlPoint3D finalStart = BuildHermitePoint(pts, ups,
start);
    HermiteControlPoint3D finalEnd = BuildHermitePoint(pts, ups,
end);

    Vector3 baseTangent = Vector3.Normalize(pts[end].Position -
pts[start].Position);
    HermiteControlPoint3D baseStart = new(
        pts[start].Position,
        baseTangent,
        0f,
        Vector3.Normalize(ups[start] - Vector3.Dot(ups[start],
baseTangent) * baseTangent));
    HermiteControlPoint3D baseEnd = new(
        pts[end].Position,
        baseTangent,
        0f,
        Vector3.Normalize(ups[end] - Vector3.Dot(ups[end],

```



```

baseTangent) * baseTangent));

    float t0 = pts[start].Time;
    float t1 = pts[end].Time;

    float lambda = 0f;
    float step = 0.25f;
    PHCurve3D seg = PHCurveFactory.CreateQuintic(baseStart, baseEnd,
t0, t1);

    for (int iter = 0; iter < 30 && lambda < 1f; iter++)
    {
        float target = MathF.Min(1f, lambda + step);
        HermiteControlPoint3D s = Interpolate(baseStart, finalStart,
target);
        HermiteControlPoint3D e = Interpolate(baseEnd, finalEnd,
target);
        PHCurve3D candidate = PHCurveFactory.CreateQuintic(s, e, t0,
t1);
        float posErr = ComputeMaxDeviation(candidate, pts, start,
end, out float oriErr);

        if (posErr <= posTol && oriErr <= oriTol)
        {
            seg = candidate;
            lambda = target;
            step = MathF.Min(step * 1.5f, 0.5f);
        }
        else
        {
            float hi = target;
            float lo = lambda;
            for (int j = 0; j < 4 && hi - lo > 1e-3f; j++)
            {
                float mid = 0.5f * (lo + hi);
                HermiteControlPoint3D ms = Interpolate(baseStart,
finalStart, mid);
                HermiteControlPoint3D me = Interpolate(baseEnd,
finalEnd, mid);
                PHCurve3D test = PHCurveFactory.CreateQuintic(ms,
me, t0, t1);
                float pErr = ComputeMaxDeviation(test, pts, start,
end, out float oErr);
                if (pErr <= posTol && oErr <= oriTol)
                {
                    seg = test;
                    lo = mid;
                }
                else
                {
                    hi = mid;

```

```

        }
    }

    lambda = lo;
    step *= 0.5f;

    if (step < 0.01f)
    {
        break;
    }
}

float finalPos = ComputeMaxDeviation(seg, pts, start, end, out
float finalOri);
if (finalPos <= posTol && finalOri <= oriTol)
{
    return seg;
}

return null;
}

private static HermiteControlPoint3D Interpolate(
    in HermiteControlPoint3D a,
    in HermiteControlPoint3D b,
    float t)
{
    Vector3 tangent = Vector3.Normalize(Vector3.Lerp(a.Tangent,
b.Tangent, t));
    float curvature = a.Curvature * (1f - t) + b.Curvature * t;
    Vector3 normal =
Vector3.Normalize(Vector3.Lerp(a.PrincipalNormal, b.PrincipalNormal, t));
    return new HermiteControlPoint3D(b.Position, tangent, curvature,
normal);
}

private static PHCurve3D OptimizeG2(PHCurve3D previous, PHCurve3D
next)
{
    if (PHCurveFactory.ValidateG2(previous, next))
    {
        return next;
    }

    HermiteControlPoint3D start = new(
        previous.Position(1f),
        previous.TangentUnit(1f),
        previous.Curvature(1f),
        previous.PrincipalNormal(1f));

```

```

        HermiteControlPoint3D end = new(
            next.Position(1f),
            next.TangentUnit(1f),
            next.Curvature(1f),
            next.PrincipalNormal(1f));

        return PHCurveFactory.CreateQuintic(start, end, next.StartTime,
next.EndTime);
    }

    private static HermiteControlPoint3D BuildHermitePoint(
        List<PointData> pts,
        Vector3[] ups,
        int index)
    {
        Vector3 tangent;
        if (index < pts.Count - 1)
        {
            tangent = pts[index + 1].Position - pts[index].Position;
        }
        else
        {
            tangent = pts[index].Position - pts[index - 1].Position;
        }

        if (tangent.LengthSquared() < 1e-8f)
        {
            tangent = Vector3.UnitX;
        }

        tangent = Vector3.Normalize(tangent);

        float curvature = 0f;
        Vector3 up = ups[index];
        Vector3 normal = up - Vector3.Dot(up, tangent) * tangent;
        if (normal.LengthSquared() < 1e-6f)
        {
            normal = Vector3.Cross(tangent, Vector3.UnitY);
            if (normal.LengthSquared() < 1e-6f)
            {
                normal = Vector3.Cross(tangent, Vector3.UnitZ);
            }
        }

        normal = Vector3.Normalize(normal);

        return new HermiteControlPoint3D(pts[index].Position, tangent,
curvature, normal);
    }

    private static Vector3[] PrepareUpVectors(List<PointData> pts)

```

```

    {
        Vector3[] ups = new Vector3[pts.Count];
        for (int i = 0; i < pts.Count; ++i)
        {
            Vector3 up = pts[i].UpVector.LengthSquared() > 1e-8f ?
pts[i].UpVector : Vector3.UnitY;
            ups[i] = Vector3.Normalize(up);
        }

        float cosThreshold = MathF.Cos(0.5f);
        for (int i = 0; i < ups.Length - 1; ++i)
        {
            if (Vector3.Dot(ups[i], ups[i + 1]) < cosThreshold)
            {
                Vector3 avg = Vector3.Normalize(ups[i] + ups[i + 1]);
                ups[i] = avg;
                ups[i + 1] = avg;
            }
        }

        return ups;
    }

    private static float ComputeMaxDeviation(
        PHCurve3D seg,
        List<PointData> pts,
        int startIdx,
        int endIdx,
        out float maxOri)
    {
        float maxPos = 0f;
        maxOri = 0f;

        float t0 = pts[startIdx].Time;
        float dt = pts[endIdx].Time - t0;
        if (dt < 1e-6f)
        {
            dt = 1f;
        }

        int samples = Math.Max(5, endIdx - startIdx + 1);
        for (int i = 0; i <= samples; ++i)
        {
            float u = i / (float)samples;
            float time = t0 + u * dt;

            // nearest sample point for position comparison
            int idx = startIdx;
            while (idx < endIdx - 1 && pts[idx + 1].Time < time)
            {
                idx++;
            }
        }
    }

```

```

    }
    Vector3 expected = pts[idx].Position;

    Vector3 pos = seg.Position(u);
    float d = Vector3.Distance(pos, expected);
    if (d > maxPos)
    {
        maxPos = d;
    }

    Vector3 up = pts[idx].UpVector.LengthSquared() > 1e-8f ?
    Vector3.Normalize(pts[idx].UpVector) : Vector3.UnitY;
    if (seg.Curvature(u) > 1e-5f)
    {
        Vector3 n = seg.PrincipalNormal(u);
        float ang = MathF.Acos(Math.Clamp(Vector3.Dot(n, up),
-1f, 1f));

        if (ang > maxOri)
        {
            maxOri = ang;
        }
    }
}

return maxPos;
}
}
}

```

Beim Homotopy-Fitting wird eine einfache Startkurve schrittweise zur endgültigen Lösung deformiert. Über einen Homotopieparameter verfolgt man einen kontinuierlichen Lösungsweg, bis die gemessenen Punkte innerhalb der Toleranzen liegen. Falls die Methode scheitert, wird auf die lokale Hermite-Interpolation zurückgegriffen.

Fitting/HeuristicFitter.cs

```

using System;
using System.Collections.Generic;
using System.Numerics;

namespace PHCurveLibrary.Fitting
{
    /// <summary>
    /// Implements heuristic segmentation with optional post smoothing.
    /// The method analyses the incoming points and splits the sequence
    /// whenever position or time gaps exceed a threshold. Each segment is
    /// then fitted using <see cref="LocalHermiteFitter"/> and consecutive
    /// segments are adjusted for <c>G2</c> continuity.

```

```

    /// </summary>
    public static class HeuristicFitter
    {
        /// <summary>
        /// Fit points using heuristic segmentation.
        /// </summary>
        public static List<PHCurve3D> Fit(
            List<PointData> points,
            float positionTolerance,
            float orientationTolerance)
        {
            return FitInternal(points, positionTolerance,
orientationTolerance, false);
        }

        /// <summary>
        /// Incremental heuristic fitting.
        /// </summary>
        public static List<PHCurve3D> FitIncremental(
            List<PointData> buffer,
            float positionTolerance,
            float orientationTolerance)
        {
            return FitInternal(buffer, positionTolerance,
orientationTolerance, true);
        }

        private static List<PHCurve3D> FitInternal(
            List<PointData> pts,
            float posTol,
            float oriTol,
            bool incremental)
        {
            List<PHCurve3D> result = new();
            if (pts.Count < 2)
            {
                return result;
            }

            Vector3[] ups = PrepareUpVectors(pts);

            float distThr = posTol * 10f;
            float timeThr = posTol * 10f;
            float angleThr = oriTol * 5f;

            int start = 0;
            PHCurve3D? prevSeg = null;
            while (start < pts.Count - 1)
            {
                int end = start + 1;
                while (end < pts.Count - 1)

```

```

        {
            float gap = Vector3.Distance(pts[end].Position, pts[end
+ 1].Position);
            float tGap = pts[end + 1].Time - pts[end].Time;
            Vector3 t0 = Vector3.Normalize(pts[end + 1].Position -
pts[end].Position);
            Vector3 t1 = Vector3.Normalize(pts[end].Position -
pts[end - 1].Position);
            float ang = MathF.Acos(Math.Clamp(Vector3.Dot(t0, t1),
-1f, 1f));
            if (gap > distThr || tGap > timeThr || ang > angleThr)
            {
                break;
            }

            end++;
        }

        HermiteControlPoint3D h0 = BuildHermitePoint(pts, ups,
start);
        HermiteControlPoint3D h1 = BuildHermitePoint(pts, ups, end);
        PHCurve3D seg = PHCurveFactory.CreateQuintic(h0, h1,
pts[start].Time, pts[end].Time);

        if (prevSeg.HasValue)
        {
            seg = OptimizeG2(prevSeg.Value, seg);
        }

        result.Add(seg);
        prevSeg = seg;
        start = end;

        if (incremental && start >= pts.Count - 1)
        {
            break;
        }
    }

    if (incremental)
    {
        pts.RemoveRange(0, start);
    }

    return result;
}

private static HermiteControlPoint3D BuildHermitePoint(
    List<PointData> pts,
    Vector3[] ups,
    int index)

```

```

{
    Vector3 tangent;
    if (index < pts.Count - 1)
    {
        tangent = pts[index + 1].Position - pts[index].Position;
    }
    else
    {
        tangent = pts[index].Position - pts[index - 1].Position;
    }

    if (tangent.LengthSquared() < 1e-8f)
    {
        tangent = Vector3.UnitX;
    }

    tangent = Vector3.Normalize(tangent);

    float curvature = 0f;
    Vector3 up = ups[index];
    Vector3 normal = up - Vector3.Dot(up, tangent) * tangent;
    if (normal.LengthSquared() < 1e-6f)
    {
        normal = Vector3.Cross(tangent, Vector3.UnitY);
        if (normal.LengthSquared() < 1e-6f)
        {
            normal = Vector3.Cross(tangent, Vector3.UnitZ);
        }
    }

    normal = Vector3.Normalize(normal);

    return new HermiteControlPoint3D(pts[index].Position, tangent,
curvature, normal);
}

private static Vector3[] PrepareUpVectors(List<PointData> pts)
{
    Vector3[] ups = new Vector3[pts.Count];
    for (int i = 0; i < pts.Count; ++i)
    {
        Vector3 up = pts[i].UpVector.LengthSquared() > 1e-8f ?
pts[i].UpVector : Vector3.UnitY;
        ups[i] = Vector3.Normalize(up);
    }

    float cosThreshold = MathF.Cos(0.5f);
    for (int i = 0; i < ups.Length - 1; ++i)
    {
        if (Vector3.Dot(ups[i], ups[i + 1]) < cosThreshold)
        {

```



```

        Vector3 avg = Vector3.Normalize(ups[i] + ups[i + 1]);
        ups[i] = avg;
        ups[i + 1] = avg;
    }
}

return ups;
}

private static PHCurve3D OptimizeG2(PHCurve3D previous, PHCurve3D
next)
{
    if (PHCurveFactory.ValidateG2(previous, next))
    {
        return next;
    }

    HermiteControlPoint3D start = new(
        previous.Position(1f),
        previous.TangentUnit(1f),
        previous.Curvature(1f),
        previous.PrincipalNormal(1f));

    HermiteControlPoint3D end = new(
        next.Position(1f),
        next.TangentUnit(1f),
        next.Curvature(1f),
        next.PrincipalNormal(1f));

    return PHCurveFactory.CreateQuintic(start, end, next.StartTime,
next.EndTime);
}
}
}

```

Diese Variante segmentiert die Punktliste heuristisch nach Abstand, Zeitsprung oder Tangentenänderung. Jeder Abschnitt wird danach mit der lokalen Hermite-Methode angepasst, G^2 -Stetigkeit wird an den Übergängen geprüft und ggf. optimiert. Die heuristische Segmentierung ist besonders schnell, kann aber bei ungünstiger Wahl der Parameter zu Qualitätsverlusten führen.

1 2 3 4 5 6 8 9 10 11 12 13 14 15 16 17 18 19 21 23 24 25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40 41 42 43 44 46 47 49 50 51 52 53 54 55 56 57 60 61 62 63 64 71
72 73 74 75 77 78 79 80 81 82 83 84 85 86 87 88 89 CODE.md

file://file-U4CKsniertzu7C3u32fvPH

7 (PDF) Identification of spatial PH quintic Hermite interpolants with ...

<https://www.academia.edu/16097877>

20 22 90 Farouki_Saitou_Tsai_1998_Least_Squares_Approximation_PH_Curvs.pdf

file://file-3GgJoUhBazvPfMomEannYM

45 Aigner_Sir_Jüttler_2007_Least_Squares_Fitting.pdf

file:///file-H6ZAVxDAX2y9f3obTRGm8P

48 67 68 Farouki_Dong_2012_PHquintic_Library.pdf

file:///file-FWacyVPXj3WqP3t1spHtHQ

58 59 69 70 76 Farouki_Albrecht_1996_C2_PH Curve_homotopy_methode.pdf

file:///file-HMpV5nFhp5heBJhj65EJUt

65 Identification of spatial PH quintic Hermite interpolants with near ...

<https://www.sciencedirect.com/science/article/pii/S0167839607001276>

66 Jaklic_et_al_2015_G2_Quintic_PH_Interpolation.pdf

file:///file-34FzQQVngom11HtvhzVAAK